PROGRAMMING FOR PROBLEM SOLVING

UNIT 2 Problem Decomposition, Functions and Pointers

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

Unit 1 Brush-Up — What Did We Learn?

Before we start Unit 2...

Let's quickly recall our Unit 1 journey!

C Fundamentals

- Variables and Data Types
- Operators and Expressions
- Input and Output

Unit 1 Brush-Up — What Did We Learn?

Decision Making

- `if`
- `if--else`
- `else if` / Else-if Ladder
- Nested `if`
- `switch`

Think!

We learned how to make a program **decide**.

But what happens when the program becomes BIG?

The Problem with a Large Program

Imagine a Program with 500 Lines of Code...

Everything is Written Together

- Input the data
- Perform calculations
- Check conditions
- Display results
- Perform another calculation
- Check more conditions
- Display more results

The Problem with a Large Program

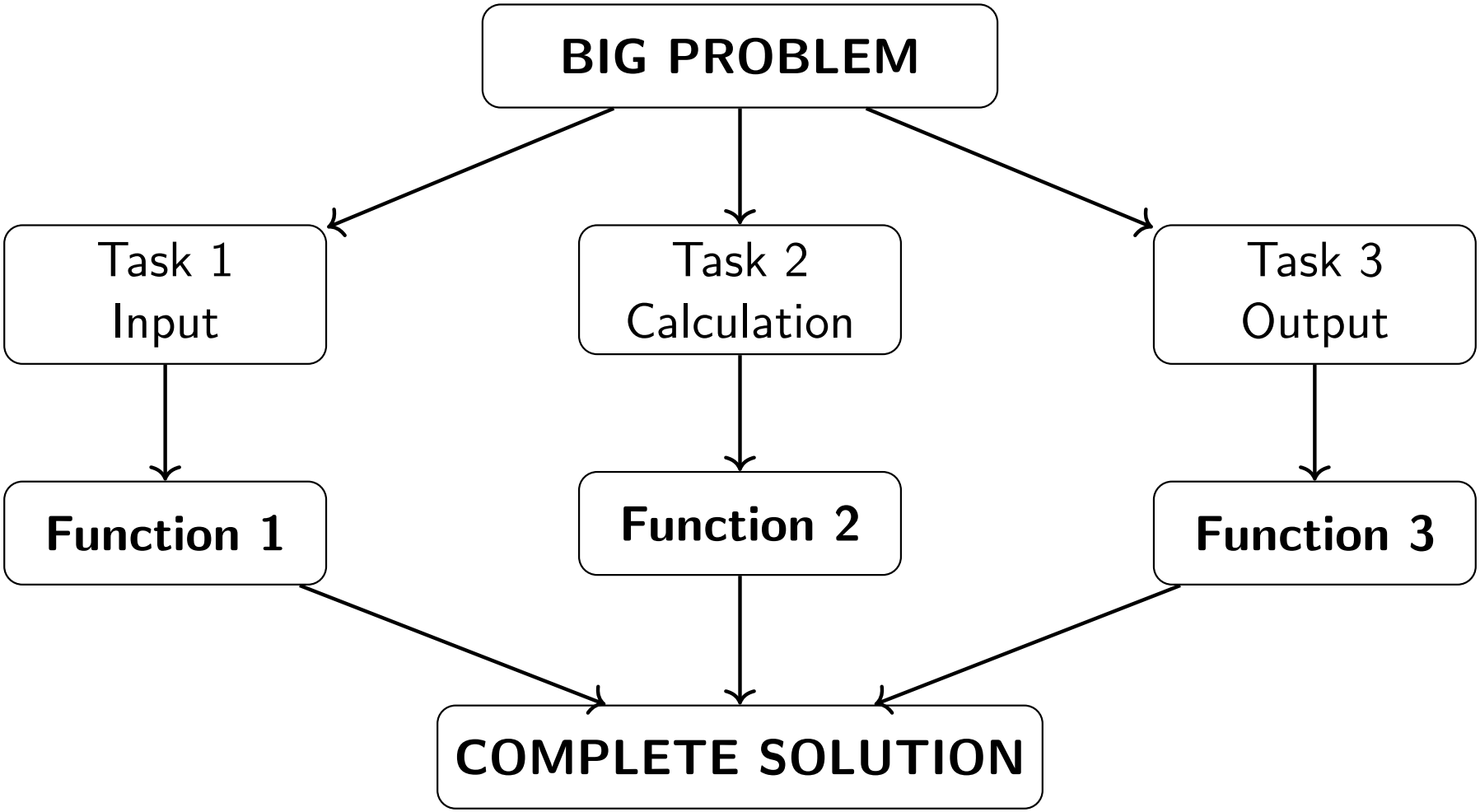
What Could Go Wrong?

- Difficult to understand
- Difficult to find errors
- Difficult to test individual tasks
- Difficult to reuse the same logic
- Difficult to maintain

Can we break this BIG problem into smaller problems?

From a Big Problem to Smaller Problems

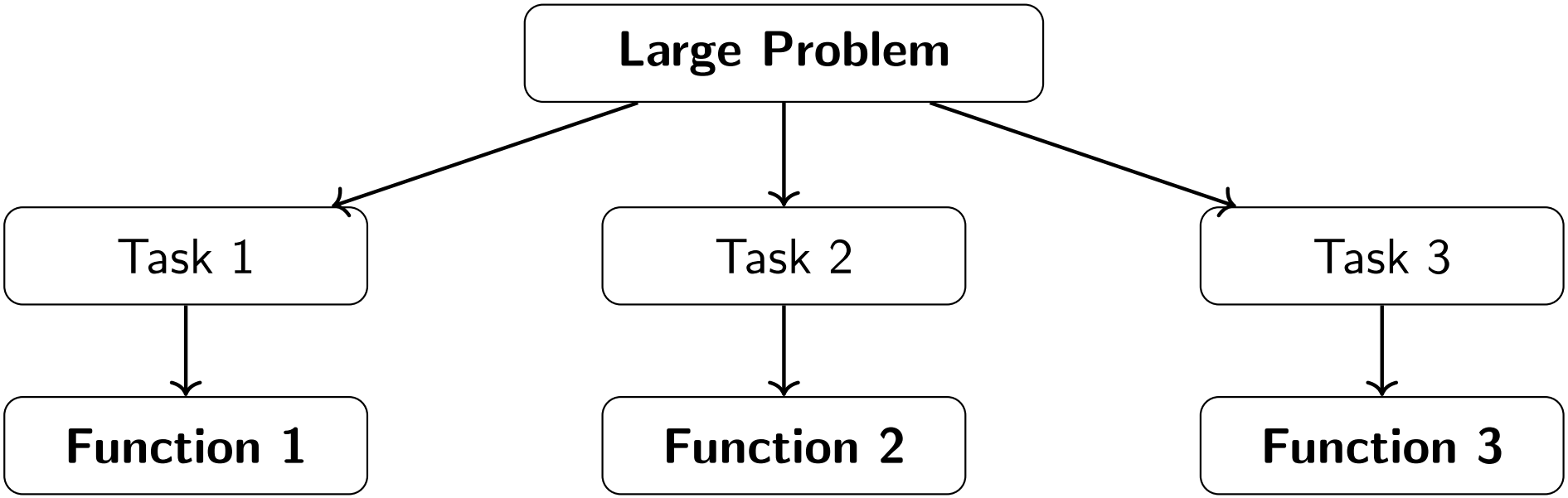
Don't Solve Everything at Once!



What is Modularity?

Definition

Modularity is the process of breaking a large programming problem into smaller, manageable and independent tasks.



Simple Idea

Big Problem → Smaller Tasks → Functions

Why Modularity?

Why Should We Divide a Program into Modules?

Without Modularity

- Large and complicated code
- Difficult to understand
- Difficult to debug
- Difficult to test
- Repeated code
- Difficult to maintain

Why Modularity?

With Modularity

- Smaller and organized code
- Easier to understand
- Easier to debug and test
- Code can be reused
- Easier to maintain
- Each module performs a specific task

Key Idea

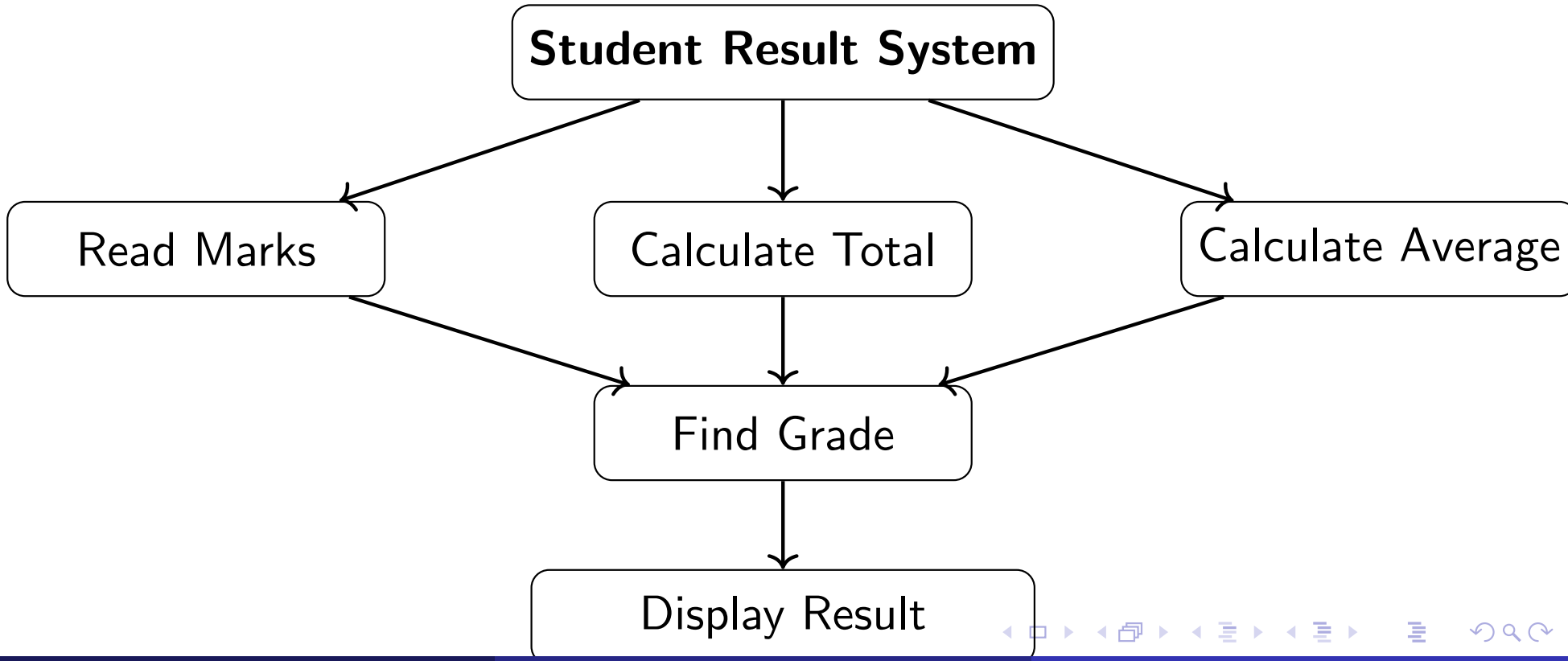
Divide the problem $\rightarrow$ Solve each part $\rightarrow$ Combine the parts

Real-Life Example of Modularity

One Big Problem

A college wants to calculate and display a student's result.

Input Marks + Calculate Result + Display Result



What is a Function?

Simple Definition

A **function** is a named block of code designed to perform a **specific task**.



Real-Life Connection

Think of a **calculator**.

$$\text{Press } + \rightarrow \text{Calculator performs addition} \rightarrow \text{Result}$$

The addition operation can be treated as a **specific task**.

Why Do We Need Functions?

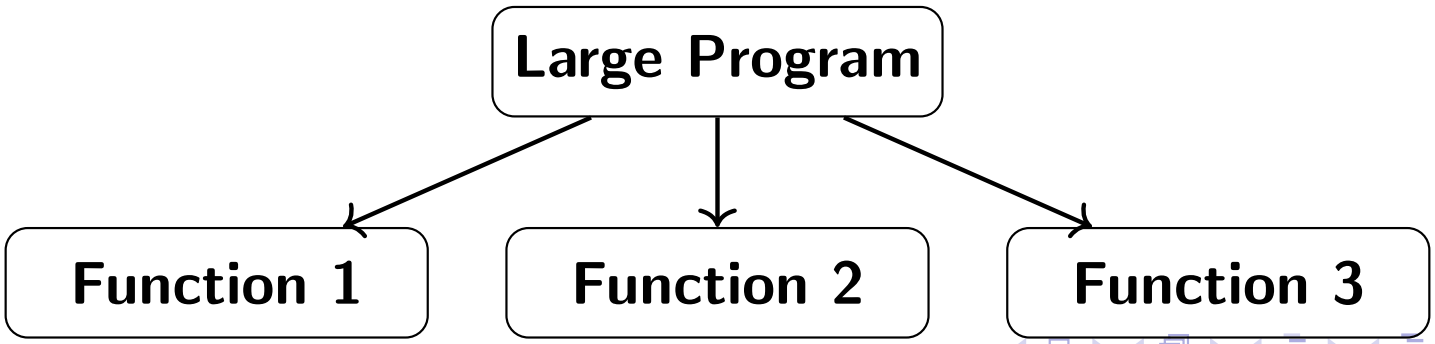
Functions Help Us Organize Our Program

Without Functions

- Everything written together
- Code becomes lengthy
- Difficult to understand
- Difficult to debug
- Repeated logic

With Functions

- Divide the program into tasks
- Code becomes organized
- Easier to understand
- Easier to test and debug
- Reuse the same task



Basic Syntax - C

Basic Structure of a Function

```
return_type function_name(parameters)
{
    // statements
}
```

return type → function name → parameters → function body

- **return type**: Type of value returned by the function
- **function name**: Name of the function
- **parameters**: Input values (if any)
- **function body**: Statements to perform the task

A function consists of:

1. **Return type** (type of value returned)
2. **Function name** (gives identity)
3. **Parameters** (input values, if any)
4. **Function body** (statements to perform the task)

Example

```
int add(int a, int b)
{
    return a + b;
}
```

1. `int` → return type (function returns an integer value)
2. `add` → function name (name of the function)
3. `int a, int b` → parameters (two integer input values)
4. `return a + b;` → statement (returns the result to the caller)



Key Idea

A function is a named block of code designed to perform a specific task.

10

How the Complete C function program?

C Program

```c
#include <stdio.h>

void greet()
{
    printf("Hello, Students!");
}

int main()
{
    greet();    // function call
    return 0;
}
```

Function Definition (creates the task)

Function Call (executes the task)

Output

```
Hello, Students!
```



How Does the Program Execute?

main() → **greet()** → **printf()** → **Return to main()**

- **main()**: Program starts from main()
- **greet()**: Control moves to the function
- **printf()**: Prints the message
- **Return to main()**: Control comes back to main()

Things to Observe

1. `greet()` is the function **definition**.
2. `greet();` inside `main()` is the **function call**.
3. The function executes only when it is called.
4. `main()` is the starting point of every C program.

How does FUNCTION execute?

Example Program

```
1  #include <stdio.h>
2
3  void greet()
4  {
5      printf("Hello!");
6  }
7
8  int main()
9  {
10     greet();
11     printf("Welcome!");
12     return 0;
   }
```

Execution Flow

1. Start from main()
Program execution begins from main().

↓

2. Function Call
greet();
Control moves to greet().

↓

3. Execute Function
Statements inside greet() are executed.

↓

4. Return to main()
Control comes back to the next statement after greet().

Output

```
Hello!
Welcome!
```

"The function runs only when it is called."

Key Points

1. main() is the starting point of every C program.
2. greet() is the function call.
3. The statements inside greet() are executed.
4. After execution, control returns to main().

Key Takeaway

A function is executed only when it is called. After execution, control returns to the calling point.

Small Functions Build Big Programs!

Parameters and Arguments

The Idea

A function can receive data from the calling function through **parameters**.

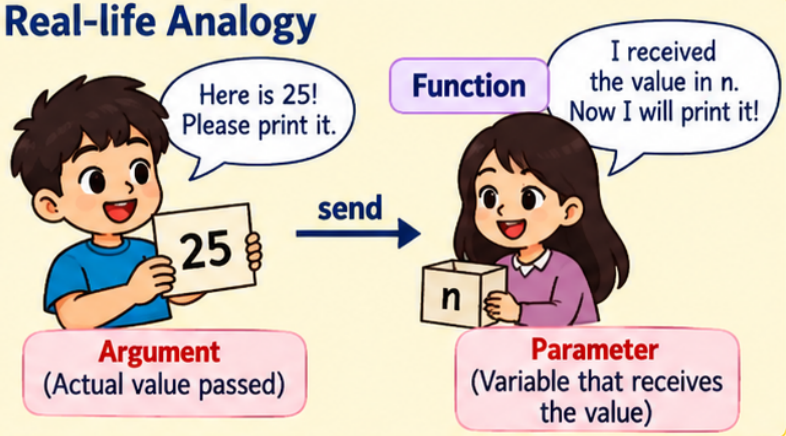
Example Program

```c
#include <stdio.h>

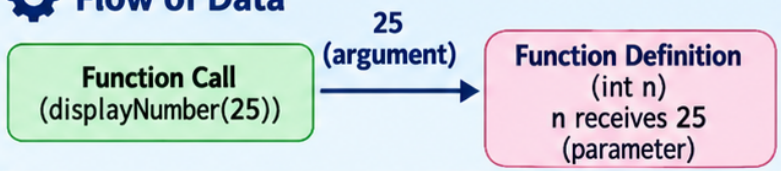
void displayNumber(int n)
{
    printf("Number = %d\n", n);
}

int main()
{
    displayNumber(25);  // 25 is an argument
    return 0;
}
```

Real-life Analogy



Flow of Data



Output

```
Number = 25
```

Key Takeaways

1. **Parameter** → Variable that receives the value in the function definition.
2. **Argument** → Actual value passed during the function call.

Different names,
Same value!
Argument goes in,
Parameter receives it.

Parameters Vs Arguments

Real-Life Analogy

Imagine ordering a pizza!



- **Parameters** — what information is expected.
- **Arguments** — the actual information provided.

Definitions

Parameter	A variable in the function definition that **receives** the value.
Argument	The **actual value** passed to the function during the function call.

Example 1 – Single Parameter

```
1  #include <stdio.h>
2
3  void greet(char name[])
4  {
5      printf("Hello %s!\n", name);
6  }
7
   greet("Arun");
```

name → Parameter (in function definition)

"Arun" → Argument (in function call)

Example 2 – Multiple Parameters

```
1  #include <stdio.h>
2
3  int add(int a, int b)
4  {
5      return a + b;
6  }
7
8  int sum = add(10, 20);  // function call
```

a, b → Parameters (in function definition)

10, 20 → Arguments (in function call)

Key Takeaways

1. **Parameter = Receiver** (variable in function definition).
2. **Argument = Actual value sent** (during function call).

Different terms... Important roles!

Function Prototype

Simple Definition

A function prototype tells the compiler about a function **before** the function is called.

Why Do We Need It?

- The compiler needs to know the **function name**, return type and parameters.
- Helps in error checking.
- Allows us to call the function before its definition.
- Improves code readability.

Key Points

1. **Prototype** declares the function.
2. **Definition** contains the actual code.
3. **Call** uses the function.

Example

```
1  #include <stdio.h>
2
3  int add(int, int);   // Function prototype
4
5  int main()
6  {
7      int result;
8
9      result = add(10, 20); // Function call
10     printf("Sum = %d\n", result);
11
12     return 0;
13 }
14
15 int add(int a, int b)  // Function definition
16 {
17     return a + b;
18 }
```

Function Prototype (tells the compiler)

Function Call (uses the function)

Function Definition (actual code)



Output

```
Sum = 30
```

What Does the Prototype Tell the Compiler?

Return Type	Function Name	Parameters
Type of value returned by the function	Name of the function	Type and number of arguments

```
int add(int, int);
```

Prototype is a promise... Definition is the work!

Three Important Parts of a Function

Prototype, Call and Definition

1. Function Prototype

```
int add(int, int);
```

Tells the compiler about the function.

2. Function Call

```
result = add(10, 20);
```

Asks the function to perform its task.

Three Important Parts of a Function

3. Function Definition

```
int add(int a, int b)
```

Contains the actual statements that perform the task.

Remember the Flow

Prototype → Call → Definition Executes → Result

Functions Based on Arguments and Return Values

Type	Arguments	Return Value	Example
1	No	No	`void display()`
2	Yes	No	`void display(int n)`
3	Yes	Yes	`int add(int a,int b)`
4	No	Yes	`int getNumber()`

Four Types of Functions in C

Think in Two Questions

1. **Does the function receive data?**
   - Yes → Arguments
   - No → No Arguments
2. **Does the function send a result back?**
   - Yes → Return Value
   - No → No Return Value

Key Idea

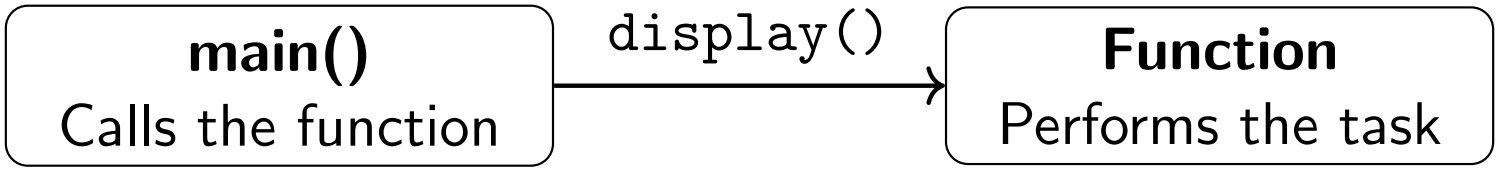
Arguments = Data going INTO the function

Return Value = Result coming OUT of the function

Type 1: No Arguments, No Return Value

What Does It Mean?

- The function does **not receive any data**.
- The function does **not return any result**.
- It simply performs a specific task.



Syntax

```
void functionName()
```

Example

```
void welcome()
```

The function simply displays a welcome message.

Solved Example 1 - Simple Display

C Program

```c
#include <stdio.h>

void welcome();

int main()
{
    welcome();   // function call
    return 0;
}

void welcome()
{
    printf("Welcome to C Programming!\n");
}
```

Function Prototype (tells the compiler)

Function Call (no arguments)

Function Definition (actual code)

Program Explanation

- The function `welcome()` does **not receive any arguments**.
- The function does **not return any value**.
- It simply displays a welcome message.
- A prototype is used because the function definition is written **after `main()`**.

Output

```
Welcome to C Programming!
```



Key Points

1. Function name: `welcome()`
2. No arguments in the function definition and call.
3. Return type is **void**.
4. The function only performs a specific task.

Real-Life Analogy



Press a doorbell.

- You don't give any data.
- It doesn't return anything.
- It just does its job (rings the bell).

Type 1 Function = Useful when you just want to perform a task!

20

Solved Example 2 — Student Information

Problem

Create a function `displayStudent()` that displays:

```
Name     : Arun
Branch   : CSE
Year     : 1
```

No data should be passed to the function.

Solved Example 2 — Student Information

Solution

```
#include <stdio.h>
void displayStudent();
int main()
{
 displayStudent();
 return 0;
}
void displayStudent()
{
 printf("Name   : Arun\n");
 printf("Branch : CSE\n");
 printf("Year   : 1\n");
}
```

Solved Example 3 — College Menu

Problem

Create a function `displayMenu()` that displays:

```
1. Student Details
2. Attendance
3. Marks
4. Exit
```

Think about why a function with **no arguments and no return value** is suitable here.

Solved Example 3 — College Menu

Solution

```
#include <stdio.h>
void displayMenu();
int main()
{
 displayMenu();
 return 0;
}
void displayMenu()
{
 printf("1. Student Details\n");
 printf("2. Attendance\n");
 printf("3. Marks\n");
 printf("4. Exit\n");
}
```

Practice Question 1

Problem

Create a function `displayCollege()` that displays the name of your college.
The function should:

- take **no arguments**
- return **no value**

Expected Output

```
ABC Engineering College
```

Think

What should be the return type of the function?

Practice Question 1 — Solution

Solution

```
#include <stdio.h>
void displayCollege();
int main()
{
 displayCollege();
 return 0;
}
void displayCollege()
{
 printf("ABC Engineering College");
}
```

Practice Question 2

Problem

Create a function `displayResultHeader()` that displays:

```
==============================
        STUDENT RESULT
==============================
```

The function should take **no arguments** and return **no value**.

Challenge

Call the function from `main()` and display the header.

Practice Question 2 — Solution

Solution

```
#include <stdio.h>
void displayResultHeader();
int main()
{
 displayResultHeader();
 return 0;
}
void displayResultHeader()
{
 printf("================================\n");
 printf("        STUDENT RESULT\n");
 printf("================================\n");
}
```

Practice Question 3

Scenario

Before an examination begins, the system should display important instructions to students.

Create a function `displayInstructions()` that displays:

- Switch off mobile phones
- Carry your ID card
- Do not leave the examination hall early

Requirement

The function must have:

No Arguments + No Return Value

Practice Question 3 — Solution

Solution

```
#include <stdio.h>
void displayInstructions();
int main()
{
   displayInstructions();
   return 0;
}
void displayInstructions()
{
   printf("EXAMINATION INSTRUCTIONS");
   printf("Switch off mobile phones");
   printf("Carry your ID card");
   printf("Do not leave the examination hall early");
}
```

Practice Question 4

Scenario

An ATM should display a welcome screen before showing the transaction options.
Create two functions:

- `displayWelcome()`
- `displayMenu()`

Both functions should take **no arguments** and return **no value**.

Expected Flow

Welcome Message $\rightarrow$ Menu

Think

Why is it better to create two functions instead of putting everything inside `main()`?

Practice Question 4 — Solution

Solution

```
#include <stdio.h>
void displayWelcome();
void displayMenu();
int main()
{
 displayWelcome();
 displayMenu();
 return 0;
}
void displayWelcome()
{
 printf("===== WELCOME TO ATM =====\n");
}
```

Practice Question 4 — Solution

Solution

```
void displayMenu()
{
 printf("1. Check Balance\n");
 printf("2. Deposit\n");
 printf("3. Withdraw\n");
 printf("4. Exit\n");
}
```

Student Portal Display

Scenario

Design a simple student portal using functions.
Create the following functions:

- `displayCollegeName()`
- `displayStudentMenu()`
- `displayFooter()`

Each function should take **no arguments** and return **no value**.

Expected Structure

College Name

↓

Student Menu

Practice Question 5

Scenario

Design a simple student portal using functions.
Create the following functions:

- `displayCollegeName()`
- `displayStudentMenu()`
- `displayFooter()`

Each function should take **no arguments** and return **no value**.

Expected Structure

College Name

↓

Student Menu

↓

Footer

Practice Question 5 — Solution

Solution

```
#include <stdio.h>

/* Function Prototypes */
void displayCollegeName();
void displayStudentMenu();
void displayFooter();

int main()
{
 displayCollegeName();
 displayStudentMenu();
 displayFooter();

 return 0;
}
```

Practice Question 5 — Solution

Solution

```
/* Function Definitions */
void displayCollegeName()
{
 printf("ABC Engineering College\n");
}
void displayStudentMenu()
{
 printf("\n===== STUDENT PORTAL =====\n");
 printf("1. View Profile\n");
 printf("2. View Attendance\n");
 printf("3. View Marks\n");
 printf("4. Exit\n");
}
```

Practice Question 5 — Solution

Solution

```
void displayFooter()
{
 printf("\n============================\n");
 printf("      Thank You!\n");
 printf("============================\n");
}
```